SQL — 100 Most Important Coding Questions

The SQL problems worth prioritizing for interviews and Data Analyst/Data Science roles, from fundamentals to advanced analytics.

SQL dialect: MySQL 8.0. Sample schema used throughout the PDFs (MySQL 8.0):
employees(employee_id, first_name, last_name, department_id, salary, hire_date, manager_id)
departments(department_id, department_name)
customers(customer_id, customer_name, city, signup_date)
orders(order_id, customer_id, order_date, amount, status)
products(product_id, product_name, category, price)
order_items(order_id, product_id, quantity)

1. SELECT + WHERE

Question: Find employees earning above 50000.

SQL:

```
SELECT first_name,salary FROM employees WHERE salary>50000;
```

Sample output: Matching employees.

2. ORDER BY + LIMIT

Question: Find top 5 salaries.

SQL:

```
SELECT first_name,salary FROM employees ORDER BY salary DESC LIMIT 5;
```

Sample output: Five highest salaries.

3. DISTINCT

Question: Find unique cities.

SQL:

```
SELECT DISTINCT city FROM customers;
```

Sample output: Unique cities.

4. COUNT

Question: Count employees.

SQL:

```
SELECT COUNT(*) FROM employees;
```

Sample output: Employee count.

5. SUM + GROUP BY

Question: Total sales by status.

SQL:

```
SELECT status,SUM(amount) total FROM orders GROUP BY status;
```

Sample output: Sales per status.

6. AVG

Question: Average salary.

SQL:

```
SELECT AVG(salary) FROM employees;
```

Sample output: Average salary.

7. HAVING

Question: Departments with at least 10 employees.

SQL:

```
SELECT department_id,COUNT(*) cnt FROM employees GROUP BY department_id HAVING cnt>=10;
```

Sample output: Qualifying departments.

8. INNER JOIN

Question: Employee with department name.

SQL:

```
SELECT e.first_name,d.department_name FROM employees e JOIN departments d ON e.department_id=d.department_id;
```

Sample output: Joined rows.

9. LEFT JOIN

Question: All customers and order count.

SQL:

```
SELECT c.customer_name,COUNT(o.order_id) cnt FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id GROU
P BY c.customer_id,c.customer_name;
```

Sample output: Every customer.

10. SELF JOIN

Question: Employee and manager names.

SQL:

```
SELECT e.first_name employee,m.first_name manager FROM employees e LEFT JOIN employees m ON e.manager_id=m.employee_
id;
```

Sample output: Employee-manager pairs.

11. Subquery average

Question: Employees above average salary.

SQL:

```
SELECT * FROM employees WHERE salary>(SELECT AVG(salary) FROM employees);
```

Sample output: Above-average employees.

12. Second highest salary

Question: Find second-highest salary.

SQL:

```
SELECT MAX(salary) FROM employees WHERE salary<(SELECT MAX(salary) FROM employees);
```

Sample output: Second-highest salary.

13. Nth highest

Question: Find 3rd highest salary.

SQL:

```
SELECT salary FROM (SELECT DISTINCT salary,DENSE_RANK() OVER(ORDER BY salary DESC) r FROM employees)x WHERE r=3;
```

Sample output: Third-highest salary.

14. Duplicate values

Question: Find duplicate customer names.

SQL:

```
SELECT customer_name,COUNT(*) cnt FROM customers GROUP BY customer_name HAVING cnt>1;
```

Sample output: Duplicate names.

15. CASE

Question: Create salary bands.

SQL:

```
SELECT first_name,CASE WHEN salary>=80000 THEN 'High' WHEN salary>=50000 THEN 'Medium' ELSE 'Low' END band FROM empl
oyees;
```

Sample output: Salary bands.

16. COALESCE

Question: Replace missing manager IDs.

SQL:

```
SELECT first_name,COALESCE(manager_id,0) manager_id FROM employees;
```

Sample output: NULL becomes 0.

17. Date filter

Question: Orders in current year.

SQL:

```
SELECT * FROM orders WHERE YEAR(order_date)=YEAR(CURDATE());
```

Sample output: Current-year orders.

18. Monthly sales

Question: Calculate monthly sales.

SQL:

```
SELECT DATE_FORMAT(order_date,'%Y-%m') mon,SUM(amount) sales FROM orders GROUP BY mon;
```

Sample output: Monthly totals.

19. Top customer

Question: Find highest-spending customer.

SQL:

```
SELECT customer_id,SUM(amount) spend FROM orders GROUP BY customer_id ORDER BY spend DESC LIMIT 1;
```

Sample output: Top customer.

20. Customers with no orders

Question: Find customers who never ordered.

SQL:

```
SELECT c.* FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
```

Sample output: No-order customers.

21. Products never sold

Question: Find never-ordered products.

SQL:

```
SELECT p.* FROM products p LEFT JOIN order_items oi ON p.product_id=oi.product_id WHERE oi.product_id IS NULL;
```

Sample output: Unsold products.

22. Revenue by product

Question: Calculate product revenue.

SQL:

```
SELECT p.product_name,SUM(oi.quantity*p.price) revenue FROM products p JOIN order_items oi ON p.product_id=oi.produc
t_id GROUP BY p.product_id,p.product_name;
```

Sample output: Product revenue.

23. Window ROW_NUMBER

Question: Number employees by salary.

SQL:

```
SELECT first_name,salary,ROW_NUMBER() OVER(ORDER BY salary DESC) rn FROM employees;
```

Sample output: Sequential ranks.

24. Window RANK

Question: Rank salaries with ties.

SQL:

```
SELECT first_name,salary,RANK() OVER(ORDER BY salary DESC) rnk FROM employees;
```

Sample output: Rank with gaps.

25. Window DENSE_RANK

Question: Rank salaries without gaps.

SQL:

```
SELECT first_name,salary,DENSE_RANK() OVER(ORDER BY salary DESC) rnk FROM employees;
```

Sample output: Dense rank.

26. Top 3 per department

Question: Find top 3 earners in each department.

SQL:

```
SELECT * FROM (SELECT e.*,ROW_NUMBER() OVER(PARTITION BY department_id ORDER BY salary DESC) rn FROM employees e)x W
HERE rn<=3;
```

Sample output: Top 3 per department.

27. LAG

Question: Compare each salary to previous salary.

SQL:

```
SELECT first_name,salary,LAG(salary) OVER(ORDER BY salary) previous_salary FROM employees;
```

Sample output: Previous salary.

28. Running total

Question: Running order sales.

SQL:

```
SELECT order_id,amount,SUM(amount) OVER(ORDER BY order_id) running_sales FROM orders;
```

Sample output: Cumulative sales.

29. Moving average

Question: 3-order moving average.

SQL:

```
SELECT order_id,AVG(amount) OVER(ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) moving_avg FROM orders;
```

Sample output: Moving average.

30. CTE

Question: Find above-average employees with a CTE.

SQL:

```
WITH a AS (SELECT AVG(salary) avg_salary FROM employees) SELECT e.* FROM employees e CROSS JOIN a WHERE e.salary>a.a
vg_salary;
```

Sample output: Above-average employees.

31. EXISTS

Question: Customers with orders.

SQL:

```
SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);
```

Sample output: Customers with orders.

32. NOT EXISTS

Question: Customers without orders.

SQL:

```
SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);
```

Sample output: Customers without orders.

33. Conditional aggregation

Question: Completed vs cancelled counts.

SQL:

```
SELECT SUM(status='Completed') completed,SUM(status='Cancelled') cancelled FROM orders;
```

Sample output: Two counts.

34. Department payroll share

Question: Employee salary percentage of department.

SQL:

```
SELECT employee_id,100*salary/SUM(salary) OVER(PARTITION BY department_id) pct FROM employees;
```

Sample output: Payroll percentage.

35. Department average

Question: Employees above department average.

SQL:

```
SELECT e.* FROM employees e WHERE e.salary>(SELECT AVG(x.salary) FROM employees x WHERE x.department_id=e.department
_id);
```

Sample output: Above department average.

36. Latest order

Question: Latest order per customer.

SQL:

```
SELECT * FROM (SELECT o.*,ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date DESC) rn FROM orders o)x WH
ERE rn=1;
```

Sample output: Latest order per customer.

37. First order

Question: First order per customer.

SQL:

```
SELECT * FROM (SELECT o.*,ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date) rn FROM orders o)x WHERE r
n=1;
```

Sample output: First order per customer.

38. Monthly growth

Question: Month-over-month sales growth.

SQL:

```
WITH m AS (SELECT DATE_FORMAT(order_date,'%Y-%m') mon,SUM(amount) sales FROM orders GROUP BY mon) SELECT mon,sales,s
ales-LAG(sales) OVER(ORDER BY mon) change_amt FROM m;
```

Sample output: Monthly change.

39. Repeat customers

Question: Customers with 2+ orders.

SQL:

```
SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*)>=2;
```

Sample output: Repeat customers.

40. Average order value

Question: Average amount per order.

SQL:

```
SELECT AVG(amount) avg_order_value FROM orders;
```

Sample output: Average order value.

41. Order count by city

Question: Count orders by customer city.

SQL:

```
SELECT c.city,COUNT(o.order_id) orders FROM customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.city
;
```

Sample output: Orders per city.

42. Product ranking

Question: Rank products by revenue.

SQL:

```
WITH r AS (SELECT p.product_id,SUM(oi.quantity*p.price) revenue FROM products p JOIN order_items oi ON p.product_id=
oi.product_id GROUP BY p.product_id) SELECT *,DENSE_RANK() OVER(ORDER BY revenue DESC) rnk FROM r;
```

Sample output: Revenue rank.

43. Median salary

Question: Calculate median salary.

SQL:

```
WITH x AS (SELECT salary,ROW_NUMBER() OVER(ORDER BY salary) rn,COUNT(*) OVER() n FROM employees) SELECT AVG(salary)
median FROM x WHERE rn IN(FLOOR((n+1)/2),FLOOR((n+2)/2));
```

Sample output: Median salary.

44. Customer lifetime value

Question: Completed revenue per customer.

SQL:

```
SELECT customer_id,SUM(CASE WHEN status='Completed' THEN amount ELSE 0 END) ltv FROM orders GROUP BY customer_id;
```

Sample output: Customer LTV.

45. Data quality nulls

Question: Count NULL salaries and departments.

SQL:

```
SELECT SUM(salary IS NULL) null_salary,SUM(department_id IS NULL) null_department FROM employees;
```

Sample output: NULL counts.

46. Data quality orphan rows

Question: Find employees with invalid department references.

SQL:

```
SELECT e.* FROM employees e LEFT JOIN departments d ON e.department_id=d.department_id WHERE d.department_id IS NULL
 AND e.department_id IS NOT NULL;
```

Sample output: Orphan rows.

47. Transaction revenue

Question: Revenue from quantity × product price.

SQL:

```
SELECT SUM(oi.quantity*p.price) revenue FROM order_items oi JOIN products p ON oi.product_id=p.product_id;
```

Sample output: Total item revenue.

48. Top city by revenue

Question: Find highest-sales city.

SQL:

```
SELECT c.city,SUM(o.amount) revenue FROM customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.city OR
DER BY revenue DESC LIMIT 1;
```

Sample output: Top city.

49. Daily active customers

Question: Unique customers per day.

SQL:

```
SELECT DATE(order_date) dt,COUNT(DISTINCT customer_id) active_customers FROM orders GROUP BY dt;
```

Sample output: Daily active customers.

50. 30-day rolling sales

Question: Rolling sales for the last 30 days.

SQL:

```
SELECT DATE(o1.order_date) dt,SUM(o2.amount) sales_30d FROM orders o1 JOIN orders o2 ON DATE(o2.order_date) BETWEEN
DATE(o1.order_date)-INTERVAL 29 DAY AND DATE(o1.order_date) GROUP BY dt;
```

Sample output: 30-day rolling sales.

51. Top 20% customers

Question: Find customers in top revenue quintile.

SQL:

```
WITH x AS (SELECT customer_id,SUM(amount) revenue FROM orders GROUP BY customer_id),y AS (SELECT *,NTILE(5) OVER(ORD
ER BY revenue DESC) q FROM x) SELECT * FROM y WHERE q=1;
```

Sample output: Top revenue quintile.

52. Consecutive activity

Question: Find customer daily streaks.

SQL:

```
WITH x AS (SELECT DISTINCT customer_id,DATE(order_date) dt FROM orders),y AS (SELECT *,dt-INTERVAL ROW_NUMBER() OVER
(PARTITION BY customer_id ORDER BY dt) DAY grp FROM x) SELECT customer_id,COUNT(*) streak FROM y GROUP BY customer_i
d,grp;
```

Sample output: Streak lengths.

53. Cohort analysis

Question: Group revenue by signup cohort and order month.

SQL:

```
SELECT DATE_FORMAT(c.signup_date,'%Y-%m') cohort,DATE_FORMAT(o.order_date,'%Y-%m') month,SUM(o.amount) revenue FROM
customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY cohort,month;
```

Sample output: Cohort revenue.

54. Hierarchy

Question: Build employee hierarchy with recursive CTE.

SQL:

```
WITH RECURSIVE org AS (SELECT employee_id,first_name,manager_id,0 depth FROM employees WHERE manager_id IS NULL UNIO
N ALL SELECT e.employee_id,e.first_name,e.manager_id,o.depth+1 FROM employees e JOIN org o ON e.manager_id=o.employe
e_id) SELECT * FROM org;
```

Sample output: Hierarchy levels.

55. Year-over-year

Question: Compare monthly sales with previous year.

SQL:

```
WITH m AS (SELECT YEAR(order_date) yr,MONTH(order_date) mon,SUM(amount) sales FROM orders GROUP BY yr,mon) SELECT a.
yr,a.mon,a.sales,b.sales prior_year FROM m a LEFT JOIN m b ON b.yr=a.yr-1 AND b.mon=a.mon;
```

Sample output: YoY comparison.

56. Anomaly detection

Question: Find orders > 3 standard deviations from mean.

SQL:

```
WITH s AS (SELECT AVG(amount) av,STDDEV_POP(amount) sd FROM orders) SELECT o.* FROM orders o CROSS JOIN s WHERE o.am
ount>s.av+3*s.sd;
```

Sample output: Outlier orders.

57. Basket pairs

Question: Find products bought together.

SQL:

```
SELECT a.product_id,b.product_id,COUNT(*) pair_count FROM order_items a JOIN order_items b ON a.order_id=b.order_id
AND a.product_id<b.product_id GROUP BY a.product_id,b.product_id ORDER BY pair_count DESC;
```

Sample output: Product pairs.

58. Final interview query

Question: Find each department's highest-paid employee(s), including ties.

SQL:

```
WITH x AS (SELECT e.*,DENSE_RANK() OVER(PARTITION BY department_id ORDER BY salary DESC) rnk FROM employees e) SELEC
T * FROM x WHERE rnk=1;
```

Sample output: Highest-paid employee(s) per department.